

Flex code:

```
%{  
/* Definition section */  
#include <stdio.h>  
#include <stdlib.h>  
#include "y.tab.h" /* Include the header file generated by Bison*/  
extern int yylval; /* `yylval` is used to store token values */  
%}  
  
/* Token Rules Section */  
%%  
0      { yylval = 0; return ZERO; } /* Recognize '0' as ZERO */  
1      { yylval = 1; return ONE; }  /* Recognize '1' as ONE */  
[ \t]  { /* Ignore whitespace */ } /* Skip spaces and tabs */  
\n     { return 0; }                /* End input on a newline */  
.  
      { return yytext[0]; }        /* Return any other character as-is*/  
%%  
  
/* Helper function for Flex */  
int yywrap() {  
    return 1; /* Signals end of input */  
}
```

Yacc code:

```
/* Definition section */
%{
/* C code section */
#include <stdio.h>
#include <stdlib.h>
extern int yylex(void); /* Declare yylex() function for bison to use */
void yyerror(const char *s) {
    fprintf(stderr, "Error: %s\n", s); /* Print error message */
}
%}

/* Token definitions */
%token ZERO ONE

/* Rule Section */
%%
N : L { printf("\nDecimal Value: %d\n", $$); }
    ;

L : L B { $$ = $1 * 2 + $2; } /* Recursive case: Combine previous L with B */
    | B { $$ = $1; } /* Base case: single B */
    ;

B : ZERO { $$ = 0; } /* Assign value 0 for ZERO */
    | ONE { $$ = 1; } /* Assign value 1 for ONE */
    ;

%%

/* Driver code */
int main() {
    printf("Enter a binary sequence (e.g., 0101): ");
    return yyparse(); /* Call the parser */
}
```

Linux:

```
sudo apt-get install flex  
sudo apt-get install bison  
flex 1.l  
yacc -d 1.y  
gcc lex.yy.c y.tab.c  
./a.out
```

Windows:

```
flex 1.l  
yacc -d 1.y  
rename 1.tab.c y.tab.c  
rename 1.tab.h y.tab.h  
gcc lex.yy.c y.tab.c  
a.exe
```